DD2482 Project Teefinder

Tobias Bjurström & Peter Byström

October 2026

1 Teefinder

The aim of the project was to implement a Devops pipeline surrounding the existing project *Teefinder*, visitable at `https://teefinder-devops.web.app/`. Teefinder is a web application that lets you find, save, and review golf courses around the world. The original project was hosted using Firebase, Google's web app development platform. Limited devops principles were in place in the original project, such as a base for testing the application, but all parts of a devops pipeline was implemented, either completely from scratch, or expanded from the limited tools in place from the original project.

The project's Github is located at `https://github.com/tobiasbjurstrom/teefinder-devops`. The devops pipeline consists of a CI/CD pipeline using Github Actions, Infrastructure as Code with Firebase, and security automation and checks using Dependabot, Github Secret scanning, and TruffleHog.

Teefinder uses the Google Maps API to display a map and to find golf courses. Pressing the map shows a list of courses close to the selected location. A user can log in using Google (currently only working when app is run locally) or use the website as a Guest. Logged in users can favorite golf courses. (More on the backend/database of users!) The final application is deployed automatically to Firebase and is visitable via the link above.

2 CI/CD Pipeline

The main CI/CD pipeline is implemented with Github Actions. There are 3 jobs defined in the Actions Workflow: Secret sniffing with TruffleHog, Test & Build using `Vite` and `Vitest`, and deployment to Firebase with `FirebaseExtended/action-hosting-deploy`. The jobs run sequentially, and if one job fails, the remaining are not run. Deployment is only done when updating the main branch, this allows pull requests to pass the checks, but only be deployed when the PR is merged.

TruffleHog is used to scan committed files for exposed secrets. It provides functionality to only alert on real keys with the flag `--results=verified`, but for testing and demonstration purposes this is currently disabled. If an exposed key is found, TruffleHog returns a failure and exits the job.

The testing part of the CI pipeline is implemented using `Veritest`. The job simply runs`npm test` which finds all files ending with `.test.jsx` and runs the tests contained in them. There are two test files, `App.test.jsx` and `CoursesView.test.jsx`. The former is a small file containing unit tests, making sure the index.html file exists and that the model initializes in the right way. The latter file uses `jest-dom` to set up and test the DOM states. Currently, it creates two dummy courses and verifies they are correctly shown and that the associated favorite buttons are rendered.

Once the tests are done, the application is built and uploaded. A final verification of the build is done, checking that the index.html file is correctly in the build folder, that the assets are there, and finally that the API key is in the final build.

The CD part of the pipeline is, as mentioned, only run when changes are done to main. Actions downloads the build artifact uploaded in the previous step, and deploys it automatically on firebase. The result is that the application is only deployed when it has passed the relevant checks and tests, and a faulty build never gets deployed but is caught before being pushed to production.

3 Infrastructure as Code

To satisfy the Infrastructure as Code requirement, we implemented Terraform using the Docker provider to define and manage the application's runtime infrastructure in version-controlled configuration files. Instead of relying on manual commands or imperative deployment scripts, the code formally declares an isolated bridge network for container segmentation, a persistent volume for caching and application data, and the application container itself with strict port bindings and runtime environment variables. All infrastructure definitions are fully parameterized using variables and environment definition files, allowing seamless environment switching and reproducibility between development and staging without code duplication. The entire workflow is integrated into our continuous integration pipeline via GitHub Actions. On every pull request and push to the repository, the pipeline runs automated syntax validation, formatting checks, and execution planning, generating a pull request comment with the projected state diff.

4 Security Automation

Security automation and was implemented in three ways: Github's built-in Secret scanning, Trufflehog, and Dependabot.

Secret scanning is a way for Github to alert the owner of a repository when it detects leaked private keys. Since it only generates alerts, not failed workflow runs, Trufflehog was also implemented. While both Trufflehog and Secret scanning detects keys, only Trufflehog has the option to verify that the keys are real. Both options are good for alerting the developers of leaked keys, but

neither hides them. This means that anyone with access to the Github repo can see the keys if they know where to look. This is a major limitation with the current implementation. To fix this, you could either implement the TruffleHog checks *before* the commit, or you could enable Push protect with GitHub Secret Protection.

The final automatic security check was implementing Dependabot. Dependabot automatically and continously scans `package.json` and `package-lock.json` to find outdated or vulnerable packages. It automatically creates a pull request updating the affected packages.

5 AI Disclosure

AI was used to get the original project running. Major rewrites were required to reduce the amount of dependencies by switching to *Vite*, and to set up the Firebase project. AI was also used in a limited capacity for error resolution, when Google searches proved insufficient.